Database

Introduction to Database Systems

- A **Database System** is an organized collection of data along with software to manage it.
- It enables **efficient storage, retrieval, and management** of data.
- Consists of:
  - **Database** (data)
  - **DBMS (Database Management System)** (software)
- Examples:
  - Student records system
  - Banking systems
  - E-commerce platforms

Introduction to Database Systems

- **Title:** Components of a Database System
- **Data:** Raw facts stored in tables (e.g., student names, IDs)
- **DBMS Software:** Tools like MySQL, Oracle, SQL Server
- **Users:**
  - End Users (students, customers)
  - Database Administrators (DBA)
  - Developers
- **Hardware:** Servers, storage devices
- **Procedures:** Rules for managing and using data

Introduction to Database Systems

- **Advantages of Database Systems**
- **Title:** Benefits of Using DBMS
- **Data Consistency:** Reduces duplication and errors
- **Data Security:** Controlled access to sensitive data
- **Data Integrity:** Accuracy and reliability of data
- **Data Sharing:** Multiple users can access data simultaneously
- **Backup & Recovery:** Protects against data loss

Database System Architecture

- Database System Architecture – Overview
- **Database System Architecture** refers to the design and structure that defines how a database system is organized and how components interact.
- It explains how **data is stored, processed, and accessed** by users and applications.
- The architecture helps ensure:
  - Efficient data management
  - Security and control
  - Scalability of the system
- Main architectures include:
  - One-Tier Architecture
  - Two-Tier Architecture
  - Three-Tier Architecture

Database System Architecture

- **One-Tier Architecture**
- **Title:** One-Tier Database Architecture
- Also called **Single-Tier Architecture**.
- The **database, DBMS, and user interface** are all on the same system.
- Users directly interact with the database.
- **Example:**
- Local database installed on a personal computer.
- **Advantages:**
- Simple to design
- Easy to use for development and testing
- **Limitations:**
- Not suitable for large or multi-user systems
- Limited security and scalability

Database System Architecture

- **Two-Tier Architecture**
- **Title:** Two-Tier Database Architecture
- Also known as **Client-Server Architecture**.
- The system has two main components:
  - **Client (Application Interface)**
  - **Database Server (DBMS + Database)**
- **How it Works:**
- Client sends requests (queries) to the server.
- Server processes the request and returns results.
- **Examples:**
- Desktop applications connected to a database server.
- **Advantages:**
- Faster communication
- Suitable for small to medium applications

Database System Architecture

- **Three-Tier Architecture**
- **Title:** Three-Tier Database Architecture
- Most commonly used architecture in modern systems.
- Consists of three layers:
  - **Presentation Layer** – User Interface (Web or App)
  - **Application Layer** – Business logic and processing
  - **Database Layer** – Database server and storage
- **Advantages:**
- Better security
- Improved scalability
- Easier maintenance and updates
- **Examples:**
- Web applications
- Online banking systems
- E-commerce platforms

Introduction to Data Models

- Data Models – Overview
- A **Data Model** is a set of concepts used to **describe the structure of a database**.
- It defines:
  - How data is stored
  - Relationships between data
  - Constraints on data
- Data models help in **designing databases efficiently**.
- They act as a **blueprint** for database development.
- **Examples:**
  Student, Course, and Enrollment relationships in a university database.

Introduction to Data Models

- **Types of Data Models**
- **Title:** Major Types of Data Models
- **Conceptual Data Model**
  - High-level view of the entire database.
  - Focuses on entities and relationships.
  - Example: Entity-Relationship (ER) Model.
- **Logical Data Model**
  - Describes data in detail without considering physical storage.
  - Defines tables, attributes, and relationships.
- **Physical Data Model**
  - Describes how data is actually stored in the system.
  - Includes indexes, storage structures, and file organization.

Introduction to Data Models

- **Common Data Models Used in DBMS**
- **Title:** Popular Data Models
- **Hierarchical Model**
  - Data organized in a tree-like structure.
  - One-to-many relationships.
- **Network Model**
  - Data organized in a graph structure.
  - Supports many-to-many relationships.
- **Relational Model**
  - Data stored in tables (rows and columns).
  - Most widely used model today.
- **Object-Oriented Model**
  - Data stored as objects similar to programming languages.

Introduction to Data Models

- **Importance of Data Models**
- **Title:** Why Data Models are Important
- Help in **clear database design**
- Improve **data consistency and accuracy**
- Make communication easier between developers and users
- Support **efficient data management**
- Provide a foundation for building complex database systems

Relational Database Concepts

- Relational Database Concepts – Overview
- A **Relational Database** organizes data into **tables (relations)**.
- Each table contains **rows and columns** to store related data.
- Proposed by **Edgar F. Codd** in 1970.
- Data in tables is connected using **relationships**.
- Widely used in modern systems like banking, education, and e-commerce.
- **Example:**
  Student Table | Course Table | Enrollment Table

Relational Database Concepts

- **Basic Elements of a Relational Database**
- **Title:** Core Components
- **Table (Relation):** Structure that stores data.
- **Row (Tuple):** A single record in a table.
- **Column (Attribute):** A field describing a property of data.
- **Domain:** Set of valid values for an attribute.
- **Example Table: Students**

Student_ID	Name	Program
101	Ali	BSCS
102	Sara	BSIT

Relational Database Concepts

- **Keys in Relational Databases**
- **Title:** Types of Keys
- **Primary Key**
  - Uniquely identifies each record in a table.
- **Foreign Key**
  - Connects one table with another.
- **Candidate Key**
  - Possible attributes that can serve as a primary key.
- **Composite Key**
  - Combination of two or more attributes used as a key.
- **Example:**
  Student_ID as Primary Key in Students table.

Relational Database Concepts

- Types of Relationships
- **One-to-One (1:1)**
  One record in a table relates to one record in another.
- **One-to-Many (1:M)**
  One record relates to multiple records.
  Example: One teacher teaches many students.
- **Many-to-Many (M:N)**
  Many records relate to many records.
  Example: Students enroll in multiple courses.
- **Importance:**
- Reduces data redundancy
- Improves data organization
- Enables efficient queries

Relational Algebra & Calculus

- **Introduction to Relational Algebra & Calculus**
- **Title:** Relational Algebra & Relational Calculus – Overview
- Both are **formal query languages** used in relational databases.
- They form the **theoretical foundation of SQL**.
- Introduced as part of the relational model by **Edgar F. Codd**.
- **Key Difference:**
- **Relational Algebra:** Procedural (specifies *how* to get the result).
- **Relational Calculus:** Non-procedural (specifies *what* result is needed).
- These concepts help understand **how database queries work internally**.

Relational Algebra & Calculus

- Relational Algebra Concepts
- A **procedural query language** used to perform operations on relations (tables).
- Uses operators to retrieve and manipulate data.
- **Basic Operations:**
- **Selection (σ):** Select specific rows
- **Projection (π):** Select specific columns
- **Union (∪):** Combine tuples from two relations
- **Set Difference (−):** Tuples in one relation but not the other
- **Cartesian Product (×):** Combine all rows from two tables
- **Rename (ρ):** Rename relation or attributes
- **Purpose:**
  Used internally by DBMS to process queries efficiently.

Relational Algebra & Calculus

- **Relational Calculus**
- **Title:** Relational Calculus Concepts
- A **non-procedural query language**.
- Focuses on **what data is required**, not how to retrieve it.
- Based on **mathematical logic and predicate calculus**.
- **Types of Relational Calculus:**
- **Tuple Relational Calculus (TRC)**
  Works with tuples (rows).
- **Domain Relational Calculus (DRC)**
  Works with attributes (columns).
- **Example Idea:**
  Retrieve students whose GPA is greater than 3.0.

Relational Algebra & Calculus

- **Comparison of Relational Algebra & Calculus**
- **Title:** Key Differences

Feature	Relational Algebra	Relational Calculus
Type	Procedural	Non-Procedural
Focus	How to retrieve data	What data is needed
Basis	Operations on relations	Predicate logic
Usage	Query execution in DBMS	Query specification

Structured Query Language (SQL)

- Structured Query Language (SQL) – Overview
- **Structured Query Language (SQL)** is the standard language used to **manage and manipulate relational databases**.
- It allows users to:
  - Store data
  - Retrieve data
  - Update data
  - Delete data
- SQL works with relational database systems such as:
  - MySQL
  - Oracle Database
  - Microsoft SQL Server
- **Common Use:**
  Used in websites, banking systems, and enterprise applications.

Structured Query Language (SQL)

- **Categories of SQL Commands**
- **Title:** Types of SQL Commands
- **DDL (Data Definition Language)**
  Used to define and modify database structure.
  Examples:
  - CREATE
  - ALTER
  - DROP
- **DML (Data Manipulation Language)**
  Used to manage data within tables.
  Examples:
  - INSERT
  - UPDATE
  - DELETE
- **DQL (Data Query Language)**
  Used to retrieve data from the database.
  Example:
  - SELECT
- **DCL (Data Control Language)**
  Used to control access to data.
  Examples:
  - GRANT
  - REVOKE

Structured Query Language (SQL)

- Simple SQL Queries
- **Create a Table**
- CREATE TABLE Students (
  Student_ID INT PRIMARY KEY,
  Name VARCHAR(50),
  Program VARCHAR(50)
  );
- **Insert Data**
- INSERT INTO Students VALUES (101, 'Ali', 'BSCS');
- **Retrieve Data**
- SELECT * FROM Students;
- **Purpose:**
  These queries help create, manage, and retrieve information from databases.

Structured Query Language (SQL)

- **Importance of SQL**
- **Title:** Why SQL is Important
- Standard language for relational databases
- Easy to learn and widely used in industry
- Supports **data analysis and reporting**
- Handles large volumes of data efficiently
- Essential skill for:
  - Database Administrators
  - Data Analysts
  - Software Developers

Advanced SQL

- Advanced SQL – Overview
- **Advanced SQL** refers to complex queries and techniques used to perform powerful database operations.
- It extends basic SQL capabilities to handle **complex data retrieval, analysis, and manipulation**.
- Commonly used in large database systems and enterprise applications.
- **Key Features of Advanced SQL:**
- Joins
- Subqueries
- Aggregate Functions
- Views and Indexes

Advanced SQL

- **Title:** Joins in SQL
- **Joins** are used to retrieve data from **multiple related tables**.
- **Types of Joins:**
- **INNER JOIN:** Returns matching records from both tables.
- **LEFT JOIN:** Returns all records from the left table and matched records from the right.
- **RIGHT JOIN:** Returns all records from the right table and matched records from the left.
- **FULL JOIN:** Returns all records when there is a match in either table.
- **Example:**
- SELECT Students.Name, Courses.Course_Name
  FROM Students
  INNER JOIN Enrollment
  ON Students.Student_ID = Enrollment.Student_ID;

Advanced SQL

- Subqueries & Aggregate Functions
- **Subquery**
- A query inside another SQL query.
- Used to filter results or perform complex operations.
- Example:
- SELECT Name
  FROM Students
  WHERE GPA > (SELECT AVG(GPA) FROM Students);
- **Aggregate Functions**
  Used to perform calculations on data:
- COUNT() – Counts records
- SUM() – Total values
- AVG() – Average value
- MAX() / MIN() – Highest or lowest values

Advanced SQL

- Views & Indexes in SQL
- **Views**
- A **virtual table** created from a query.
- Simplifies complex queries and improves security.
- Example:
- CREATE VIEW Student_View AS
  SELECT Name, Program
  FROM Students;
- **Indexes**
- Improve the **speed of data retrieval**.
- Created on columns frequently used in searches.
- Example:
- CREATE INDEX idx_student_name
  ON Students(Name);

Database Design & Normalization

- Database Design – Overview
- **Database Design** is the process of structuring a database to store data efficiently.
- Goals of good design:
  - Reduce **data redundancy**
  - Improve **data integrity**
  - Ensure **efficient queries** and updates

Database Design & Normalization

- **Key Steps in Database Design:**
- Requirements Analysis – Understand what data is needed
- Conceptual Design – Use **Entity-Relationship (ER) diagrams**
- Logical Design – Map ER diagram to tables
- Physical Design – Define

Transaction Management

- **What is a Transaction?**
  - A logical unit of work that must be either **fully completed** or **fully aborted**.
  - It consists of one or more SQL operations (e.g., INSERT, UPDATE, DELETE) that form a single, logical task.
  - **Example:** Transferring $100 from Account A to Account B. This involves two steps:
    - Debit $100 from Account A.
    - Credit $100 to Account B.
- **Why is it Important?**
  - To maintain the accuracy and reliability of data in a multi-user environment.
  - To prevent data corruption from system failures, crashes, or concurrent access.
- **Visual Suggestion:** A simple diagram showing a "Start Transaction" box pointing to two database operations (Debit A, Credit B) and then pointing to a "Commit" box.

Transaction Management

- **Key SQL Commands:**
  - BEGIN TRANSACTION**:** Marks the starting point.
  - COMMIT**:** Makes all changes made by the transaction permanent in the database. The transaction ends successfully.
  - ROLLBACK**:** Aborts the transaction and undoes all changes made since the BEGIN. The database is restored to its state before the transaction started.

Transaction Management

- **Simplified State Diagram:**
  - **Active:** The initial state; the transaction is executing.
  - **Partially Committed:** After the final statement has been executed, but data is not yet permanently saved.
  - **Committed:** All changes are successfully saved to the database. (Success!)
  - **Failed:** An error occurs, or the transaction is aborted.
  - **Aborted (Rollback):** The database has been restored to its state before the transaction began.
- **Visual Suggestion:** A flow diagram: **Active** $\rightarrow$ **Partially Committed** $\rightarrow$ **Committed**. An arrow from **Active** pointing down to **Failed**, and then to **Aborted**.

Transaction Management

- The Danger of Simultaneous Access
- **Content:**
  - **The Problem (Lost Update):**
    - Imagine two people (T1 and T2) reading the same bank balance ($100) at the exact same time.
    - T1 adds $50 (calculates $150) and updates the database.
    - T2, unaware of T1's update, subtracts $20 from the original $100 (calculates $80) and updates the database.
    - **Result:** The final balance is $80, not $120 ($100 + $50 - $20). T1's update is "lost."
-

Transaction Management

- **The Solution: Locking**
  - Databases use locks to prevent this.
  - **Shared Lock (Read Lock):** Allows multiple transactions to read but not modify the data.
  - **Exclusive Lock (Write Lock):** Allows a transaction to read and modify data. While this lock is held, no other transaction can read or write the locked data.
- **Visual Suggestion:**
- A timeline diagram showing T1 and T2 operating on the same data, leading to the wrong final result.
- An arrow pointing to a "Lock" icon that prevents the second user from seeing the old data.

Concurrency Control

Definition: Mechanism to manage simultaneous operations on a database without conflicts.

Goal: Ensure data consistency, integrity, and isolation in multi-user environments.

Why needed:

- Multiple users may read/write the same data simultaneously.
- Prevents problems like lost updates, dirty reads, uncommitted data.

Key Techniques

Lock-Based Protocols

Shared (read) locks and exclusive (write) locks.

Example: Two-Phase Locking (2PL).

Timestamp-Based Protocols

Each transaction gets a timestamp.

Ensures older transactions are prioritized.

Optimistic Concurrency Control

Assumes conflicts are rare.

Validates transactions before committing.

Benefits & Challenges

Benefits:

Maintains data integrity.

Supports parallelism for better performance.

Avoids deadlocks (if designed carefully).

Challenges:

Complexity in implementing protocols.

Possible deadlocks or transaction aborts.

Overhead in high-concurrency environments.

Introduction to Recovery Management

Definition: Mechanism to restore a database to a consistent state after failures.

Goal: Ensure durability and reliability of transactions.

Why needed:

System crashes, hardware failures, or software errors can corrupt data.

Protects against lost transactions and data inconsistency.

Key Techniques

Transaction Logging

Record all changes in a log file before applying them to the database.

Enables undo (rollback) and redo operations.

Checkpoints

Periodically save database state to minimize recovery time.

Only need to process transactions after the last checkpoint.

Shadow Paging

Keep a copy of the database; changes applied to a new “shadow” version.

If failure occurs, revert to the shadow copy.

Basics of Indexing & File Organization

- **File Organization**: How data is physically stored in files.
- **Heap File:** Unordered storage, simple but slow for searches.
- **Sequential File:** Sorted by key, faster search, slower insert/delete.
- **Indexing:** Helps locate data quickly without scanning the whole file.
- **Primary Index:** Based on the main key of the table.
- **Secondary Index:** Based on non-key attributes for faster search.
- **Goal:** Reduce I/O operations and improve query performance.

Index Types & Structures

Single-Level Index: Simple list mapping keys → record locations.
Multi-Level Index: Index on top of index for large files.
Tree-Based Index (B+ Tree):
Balanced tree structure for efficient search, insert, delete.
All leaf nodes linked for range queries.
Hash-Based Indexing:
Uses hash function for direct access.
Very fast for equality queries but poor for range queries.

Trade-Offs & Optimization

Trade-Offs:

Indexes speed up read queries but slow down inserts/updates/deletes.

Extra storage overhead for indexes.

File Organization Optimization:

Clustered Index: Physically reorder records for faster access.

Non-Clustered Index: Separate structure, keeps records in original order.

Performance Considerations:

Use B+ Trees for range queries.

Use hash indexing for point queries.

Periodic reorganization to reduce fragmentation.

Basics of Query Processing

Query Processing: Converting a high-level query (SQL) into a sequence of operations.

Key Steps:

Parsing & Translation: SQL → relational algebra.

Execution: Run the operations to get results.

Example: SELECT * FROM Students WHERE age > 18;

Scan the table → filter rows → return results.

Goal: Ensure correct results with minimal effort.

Query Optimization Techniques

Heuristic Optimization:

Push selections and projections down the query tree.

Example: Filter early to reduce intermediate results.

Cost-Based Optimization:

Use table statistics to choose the lowest-cost execution plan.

Evaluate multiple plans for joins and operations.

Join Optimization:

Strategies like nested loop join, hash join, sort-merge join.

Proper join order significantly improves performance.

Advanced Challenges & Strategies

Challenges:

Large datasets → exponential number of possible plans.

Dynamic data distributions → optimal plan changes over time.

Complex queries with nested subqueries, aggregations, or multiple joins.

Advanced Strategies:

Index usage: Speed up search and joins.

Query rewriting: Transform queries to more efficient forms.

Materialized views & caching: For repeated query patterns.

Parallel execution & partitioning: Distribute workload across CPUs/servers.

Database Security

Definition: Protecting database data from unauthorized access or misuse.

Key Concepts:

Authentication: Verifying user identity (username/password).

Authorization: Granting permissions (read, write, update).

Integrity: Ensuring data is accurate and consistent.

Example: Only admin can delete records; regular users can read data.

Goal: Prevent unauthorized access and accidental data loss.

Security Mechanisms

Access Control Models:

Discretionary Access Control (DAC): Users control access to their data.

Mandatory Access Control (MAC): System-enforced rules based on classification levels.

Role-Based Access Control (RBAC): Assign permissions based on roles.

Encryption:

Encrypt sensitive data at rest (stored in database) and in transit (during communication).

Auditing & Monitoring:

Track user activity, detect suspicious behavior.

Example: Log failed login attempts or unauthorized updates.

Basics of Distributed Databases

Definition: A database that is stored across multiple locations but appears as a single database to users.

Key Features:

Transparency: Users don’t need to know data location.

Replication: Copies of data stored in multiple sites for reliability.

Fragmentation: Data divided into smaller pieces (horizontal/vertical).

Example: Bank system where customer data is stored in different cities but accessed centrally.

Components & Challenges

Components:

Distributed DBMS (DDBMS): Manages data across sites.

Network Communication: Ensures nodes exchange updates correctly.

Concurrency Control: Prevent conflicts across multiple sites.

Challenges:

Data Consistency: Ensure updates in one site reflect in all sites.

Transaction Management: Distributed transactions need atomicity across nodes.

Query Processing: Optimizing queries over multiple locations.

Advanced Issues & Optimization

Advanced Problems:

Replication & Synchronization: Keeping multiple copies consistent.

Fault Tolerance: System must handle node crashes or network failures.

Data Distribution Strategies: Horizontal, vertical, or hybrid fragmentation.

Optimization Techniques:

Distributed Query Optimization: Minimize data transfer between nodes.

Two-Phase Commit Protocol (2PC): Ensures atomic transactions across sites.

Caching & Local Indexing: Speed up query access on distributed nodes.

Introduction to NoSQL

Definition: Non-relational databases designed for large-scale, distributed data.

Why NoSQL:

Handles unstructured/semi-structured data (JSON, XML).

Scales horizontally across multiple servers.

Types of NoSQL Databases:

Document Store: e.g., MongoDB – stores JSON-like documents.

Key-Value Store: e.g., Redis – simple key-value access.

Column Store: e.g., Cassandra – stores columns efficiently.

Graph Database: e.g., Neo4j – stores nodes & relationships.

Key Features & Advantages

Key Features:

Schema-less: Flexible structure, no fixed schema needed.

High Availability: Supports replication and clustering.

Eventual Consistency: Prioritizes availability over strict consistency (CAP theorem).

Advantages:

Handles big data efficiently.

Supports real-time analytics and high-throughput applications.

Easy horizontal scaling across multiple servers.

Challenges & Optimization

Challenges:

Consistency vs. Availability: Trade-offs in distributed systems (CAP theorem).

Query Limitations: Complex joins and transactions are harder than in SQL.

Data Modeling: Requires designing for access patterns, not just structure.

Advanced Strategies:

Sharding/Partitioning: Distribute large datasets across multiple nodes.

Caching & Indexing: Speed up query performance.

Hybrid Models: Some modern databases (e.g., ArangoDB) combine SQL + NoSQL features.

Basics of Data Warehousing & Mining

Data Warehouse (DW): Central repository that stores integrated data from multiple sources.

Purpose: Support decision-making and business intelligence.

Key Features:

Subject-oriented: Organized by business areas (sales, customers).

Integrated: Combines data from multiple systems.

Time-variant: Maintains historical data.

Data Mining: Process of extracting patterns and knowledge from large datasets.

Techniques & Components

Data Warehouse Components:

ETL (Extract, Transform, Load): Moves data from source → warehouse.

Metadata: Data about data; helps understand warehouse structure.

OLAP (Online Analytical Processing): Enables fast multidimensional queries.

Data Mining Techniques:

Classification: Categorize data into predefined classes.

Clustering: Group similar data items together.

Association Rule Mining: Discover relationships (e.g., Market Basket Analysis).

Regression: Predict numerical values.

Challenges & Optimization

Challenges:

Handling large-scale data efficiently.

Ensuring data quality and consistency.

Choosing the right mining algorithm for the problem.

Preserving privacy and security in mining sensitive data.

Optimization Strategies:

Use dimensional modeling (star/snowflake schema) for faster queries.

Precompute aggregates and summaries for OLAP.

Apply parallel and distributed mining algorithms for big data.